CSC111H Project Report: Harmonify

Mert Rassa, Salih Atamert Cam, Ece Avşar

Sunday, March 30, 2025

1 Introduction

From ancient times to present, humans have always yearned for social interaction. Whether for education, friendship, or relationships, we are inherently social beings. Throughout the years, this interaction has taken on all sorts of form. While in the past, people could only interact with others from far away through letters, nowadays you can interact with someone thousands of kilometers away, with only one simple click. Through communicative online platforms, people can become friends really easily. Music is one of the other main ways to connect people. It is a strong sense that helps people have fun and communicate even without speaking!

So, what about a program that combines these concepts?

Meet Harmonify!

If you are a "musicophile" and looking to connect with others who share your taste in music, and you are also tired of listening to the same songs on repeat, then "Harmonify" is the perfect place to find your underlying "rhythm" with music and other people! With our project "Harmonify", you can expand your music taste and meet with people that have your same music taste! Inspired by Spotify, which is a digital music platform that gives users access to many different songs from different genres created by inspiring artists all over the world, Harmonify makes this platform more human-interactive by recommending different types of songs to a user depending on their choice of song duration, genre, and their favorite songs. Our project also includes matching / suggesting the user to other users who are most interested in similar songs. By integrating a user-recommendation tool in our project, our project aims to connect people in terms of their similar music interests and create more connections, and aims to answer the question: **Based on preferred genre, top 5 favorite songs and preferred song duration, how can we recommend new songs and new friends to a user?**

2 Data sets

This project uses the Abstract Data Type (ADT) Graph. The graph we implemented has additional features that separates it from the Graph structure we have covered in class, which will be mentioned more in depth in section **3**. In this project, we have used a graph type also known as "Knowledge Graphs". This graph uses two datasets consisting of certain variables for different observations. Two sample observations for a song and a certain user can be found below: Table 1, Table 2. Table 2 is extracted from: https://www.kaggle.com/datasets/thedevastator/spotify-tracks-genre-dataset

username	**name**	**age**	**province**	**song_id**
GalaxyDestroyer	Ece	19	Toronto	1
GalaxyDestroyer	Ece	19	Toronto	2

Table 1: A sample from user dataset

track_id	**track_name**	**artists**	**duration_ms**	**track_genre**
1	Unholy (feat. Kim Petras)	Sam Smith;Kim Petras	156943	dance

Table 2: A sample from songs dataset

2.1 Data Wrangling/ Obtaining:

Song Dataset

The dataset obtained from kaggle included many different variables regarding the songs that were not required for conducting the project, such as "energy", "instrumentalness" and "key". Therefore, we have filtered our dataset using the **pandas** library, where functions such as DataFrame.drop() and DataFrame.groupby() were used to sort the data and extract the cruical information.

The dataset also consisted of almost 90,000 songs. Hence, using this whole dataset would increase the runtime complexity and make it more difficult for finding users to match with the input user due to less probability. When the number of songs increases, the possibility of compared users having the common song decreases thus they match with less songs with a much lower similarity score. There were also songs included in our dataset that were not really known. To overcome these problems, the dataset was ranked by "popularity" column that was included in the original dataset, and a subset of our original data with 200 songs were extracted through this code:

```
df = pd.read_csv('all_songs.csv')

# Remove duplicates (keep the first occurrence)
df = df.drop_duplicates()
df_sorted = df.groupby('track_name', sort=False).first().reset_index()

# Sort by 'popularity' in descending order and keep the top 200
df_sorted = df_sorted.sort_values(by='popularity', ascending=False).head(200)

# Drop unnecessary columns from the sorted dataframe
df_sorted = df_sorted.drop(
    columns=['track_id', 'album_name', 'explicit','danceability', 'popularity',
             'energy', 'time_signature', 'tempo', 'valence','liveness', 'instrumentalness', 'acousticness',
             'speechiness', 'mode', 'key', 'loudness'],
    errors='ignore'  # Ignore if any column is missing
)

if 'Unnamed: 0' in df.columns:
    df_sorted = df_sorted.drop('Unnamed: 0', axis=1)


# Save the sorted CSV to a new file
df_sorted.to_csv('songs_by_popularity.csv', index_label='Rank', header=False)
```

- To create "track_id" for each song based on their popularity ranking, we have started the index from 1 and incremented for each row up until 200 to get a cleaner dataset.
- After these filterings, we stored `df_sorted` within the csv file: `songs_by_popularity.csv`

User Dataset

Creating our user dataset would have required conducting a survey in which participants provided a chosen username for our program, their name, their ten favorite songs, and their current province of residence. This survey would have needed to run over an extended period to gather sufficient user input. However, due to time constraints on our project, we decided to create a synthetic user dataset by asking ChatGPT. Our prompt to extract this user dataset was as follows:

Create a user dataset with 200 users in the format: "username, name, age, province in Canada, random number from 1 to 200, and every username should have 10 lines associated with 10 random discrete numbers. We have also added our `users_small.csv` as a reference.

We then stored our full user dataset within **user_data_all_100.csv** and **user_data_all_200.csv**. For the first dataset, we only included up to 100 songs and for the second dataset, we only included 200 songs to see how the probability of our program's song recommendation changes as the song amounts change. We have observed that

with 200 dataset, our program gave approximately 2 song matches whereas with the 100 dataset, our program gave approximately 3 song matches with each launch.

The range of songs used within the program can be changed through a minor adjustment within the code below, where the function called is **compute_algorithm()** in **main.py**:

```
visualize_graph(load_review_graph('all_user_data_200_songs.csv', 'songs_by_popularity.csv'))
g = load_review_graph('all_user_data_200_songs.csv', 'songs_by_popularity.csv')
```

Here, changing the first parameter of **load_review_graph** from all_user_data_200_songs.csv to all_user_data_200_songs.csv will increase the chance of getting more songs in common with the input user and the matched user.

3 Computational Overview:

3.1 Knowledge Graphs:

Knowledge Graphs are graphs that show the interconnection between certain entities and their relationship. They consist of an "entities" vertex and each "entities" vertex is connected with a relationship, and the connections of each vertex through similar relationships are studied. Hence, knowledge graphs put the data in context by linking semantic data with entities.

The graph we have constructed consist of two different kinds of vertices: Song and User. These vertices will be distinguished by their "kind" attribute, which is either "song" or "user". In this context, users can be considered "entities" whereas we will be studying the "relationship" between different entities through a semantic data, which are "songs".

Additionally, the graph we have implemented is called a **bipartite graph**, where no user vertex is adjacent to the song vertex. Hence, every neighbour of user vertex is song vertex and every neighbour of the song vertex is the user vertex.

Vertices:

- Song vertices have attributes "item" which is a list that includes [Song's name, Song's artist, Song's duration, Song's genre] and a "neighbours" set that includes adjacent vertices which represent the Users that listen to that song. In the Graph, Songs have numerical IDs as their key to be called from '_Vertices' private dictionary. In addition, in our music data set, we sorted songs by popularity in descending order and gave them id numbers from 1 to 200. So, the song with the highest popularity score has ID = 1.
- User vertices have attributes "item" which is a list that includes [User's username, User's name, User's age, User's province] and a "neighbours" set that includes the User's adjacent vertices which represent the songs that users listen to. In the graph, Users have their usernames as key to be called from the "_Vertices" private dictionary.

3.2 Computations:

Similarity score: Our algorithm takes into consideration our implications and interpretations of compatibility score based on user input, and also takes into account the number of edges they both users share. The explanation of our implementation of the similarity score can be found below:

```
"""
Similarity score is calculated by finding;
        - the number of common songs listened by this user and the other user,
        - the number of songs that 1) this user and the other user listens and
        2) matches with this user's favourite genre.
        - the number of songs that 1) this user and the other user listens and
        2) duration of the song matches with this user's preferred duration.

        after calculating these values, we find the similarity_score by
        "number_of_common_songs * 3 + number_of_common_genres * 2 number_of_common_duration * 1" formula
"""
```

Compatible user: We implemented **compatible_user_rec_songs()** function. This function returns new user's most compatible user, the songs they listen in common, the songs only the most compatible user listen, and their matched similarity score.

1. First we created an empty dictionary for all users in the graph and an empty list for the scores. Then, we check every vertex in the graph and if the vertex is the kind "user" and different from our input user, we get the similarity score of the new user and the user that was already in the graph with the **similarity_score()** function. We store that in the score variable.

2. If the score is higher than 0, we check if there is already a key with the value of the score in the dictionary. If there is not, we create a key with the score value and that key maps to the user with that score. If there is, we add the user in the graph to the list that our score key maps to.

3. After checking every vertex, sort the scores list in ascending order then user the **.pop()** method to get the highest score. We then use this max score as a key to get the most compatible user from the users dictionary (We extract the index 0 because the program only requires 1 compatible user).

4. So now we have the most compatible user's vertex and we use the item attribute to get the parameters of the compatible user.

5. We then create 3 empty lists as : recommended_songs both_songs_you_listen songs_user_listen

6. Then we put the neighbours of the new user to the songs_user_listen, then we check neighbours of the compatible user, if they are listening a song in common we add the song to the both_songs_you_listen. If not we add song to the recommended_songs.

7. Finally, we return the most compatible users' information, both_songs_you_listen, recommended_songs, and the compatibility score.

Example Graph:

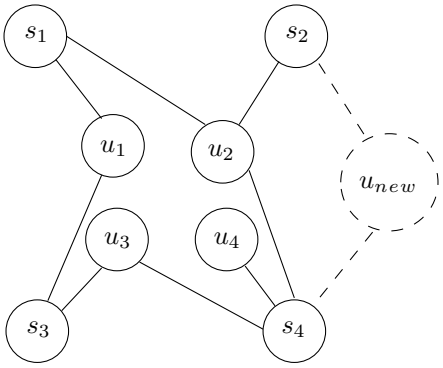

Figure 1: New user added

In Figure 1, we can see that there are already registered users (u_1, u_2, u_3, u_4) and the songs they listen to(s_1, s_2, s_3, s_4). Alongside that, when the new user starts the program, we are going to ask for their username to create a new "temporary" vertex, and top 5 songs they would like to listen to create edges between the songs and the new user. Our example graph in Figure 1 shows that the new registered user u_{new} who started the program and the edges between (u_{new} and s_2) and (u_{new} and s_4) show that the new user indicated s_2 and s_4 for their top songs. After making the new additions to the graph, we can see that the new user is listening to the two same songs (s_2 and s_4) with u_2 and the one same song (s_4) with u_4. By analyzing the genre and duration of the songs, we are going to create a compatibility score for u_2 and u_4. Finally, we are going to return the most compatible user's information (age, name, username, and province) to our new user, and songs that the most compatible user listens to which align with the new user's preferred genre choice, and the songs they both listen in common, and their matched percentage, rounded up.

3.3 Python Libraries:

Tkinter: To visualize our computation and make our project user-friendly, we will be using the Python library **tkinter**. Tkinter is a Python library used to create GUI.

When the program is run, users will be asked their preferred username, IDs of their top 5 favorite songs from the song list (There is "Song List" button that opens a PDF file that displays ID of the song, name of the song and the artist of the song in each line.), their favorite music genre (Users can look up possible music genre types from the "info" section on top left of the screen.), and preferred music duration. After filling out every input, when they click Launch, the program will display the most compatible user's username, name, age, province, match score, songs in common and recommend matched user's songs that our user didn't mention. Additionally, there is a "Visualize Graph" button that represents what our data base looks like.

Networkx: We are working with Graph Abstract Data Type, so to understand and visualize what our data looks like after we processed our data to graph, we used Networkx Python Library. In our visualized graph, green vertices represent song vertices and purple vertices represents the user vertices, and connection between purple and green vertices represents the edge.

Pandas:

As also mentioned in **2.1**, our dataset wrangling is solely dependent on the pandas library. We have used several functions to clean our data and build up our data according to our certain needs. Initially, the function **df.read_csv()** was used to open our csv file. Moreover, in terms of filtering, **df.sort_values()** was used to sort the dataset in terms of "popularity", and **df.drop()** function was used to get rid of the variables we did not need for the project.

4 Instructions for Use:

We have shared our datasets via the course email address: csc111-2025-01@cs.toronto.edu, through both UofT OneDrive and as a zip file. It was sent by Atamert. Start by opening up the requirements.txt file and installing all the libraries required. Then, open the main.py file and run the main code block. You should be seeing our user-interface like so:

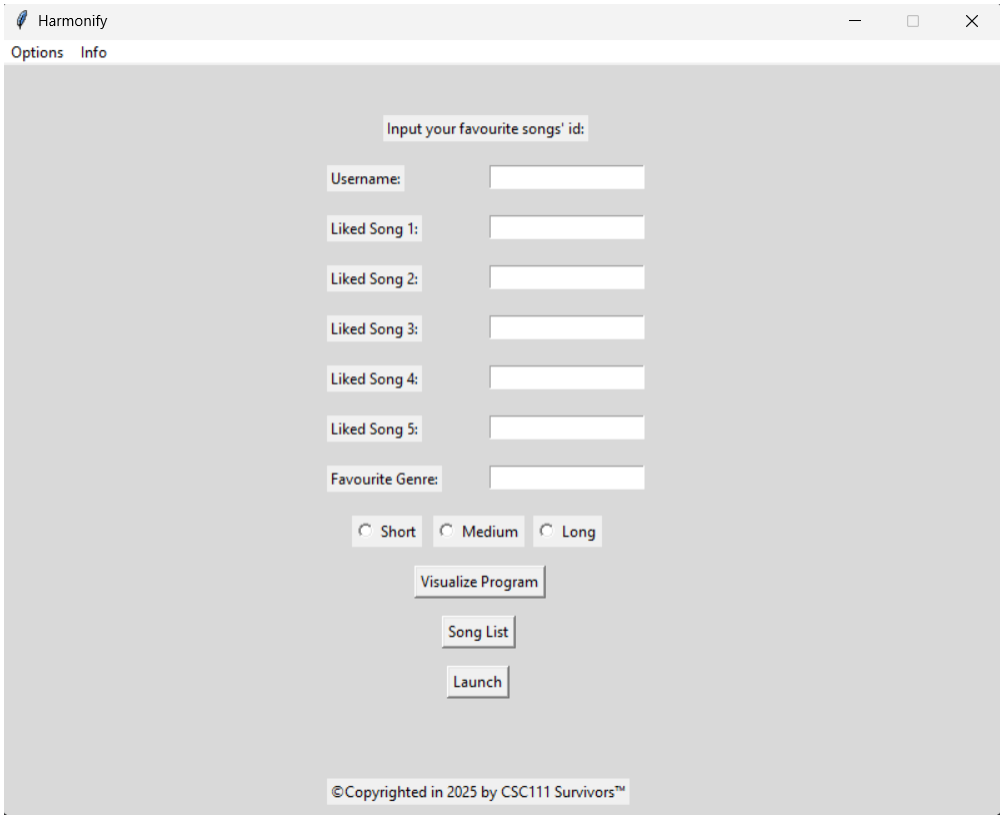

Figure 2: Program interface you should see when you first run main.py

After seeing our program interface, the user is directed to input their username, their 5 favorite songs' ids, their favorite genre, and their preferred song duration. A sample user with their inputs can be seen below:

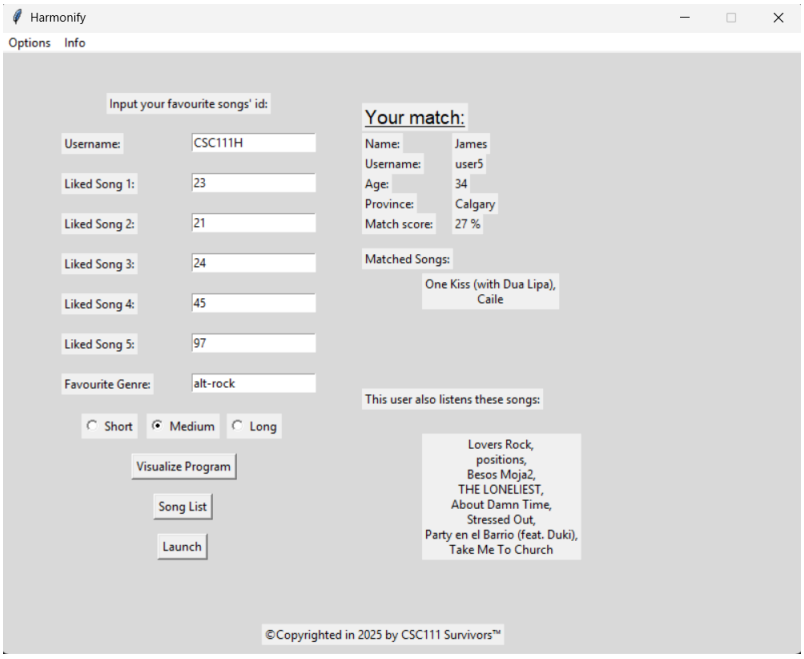

Figure 3: A sample Program interface when the program is launched after the required parameters are entered

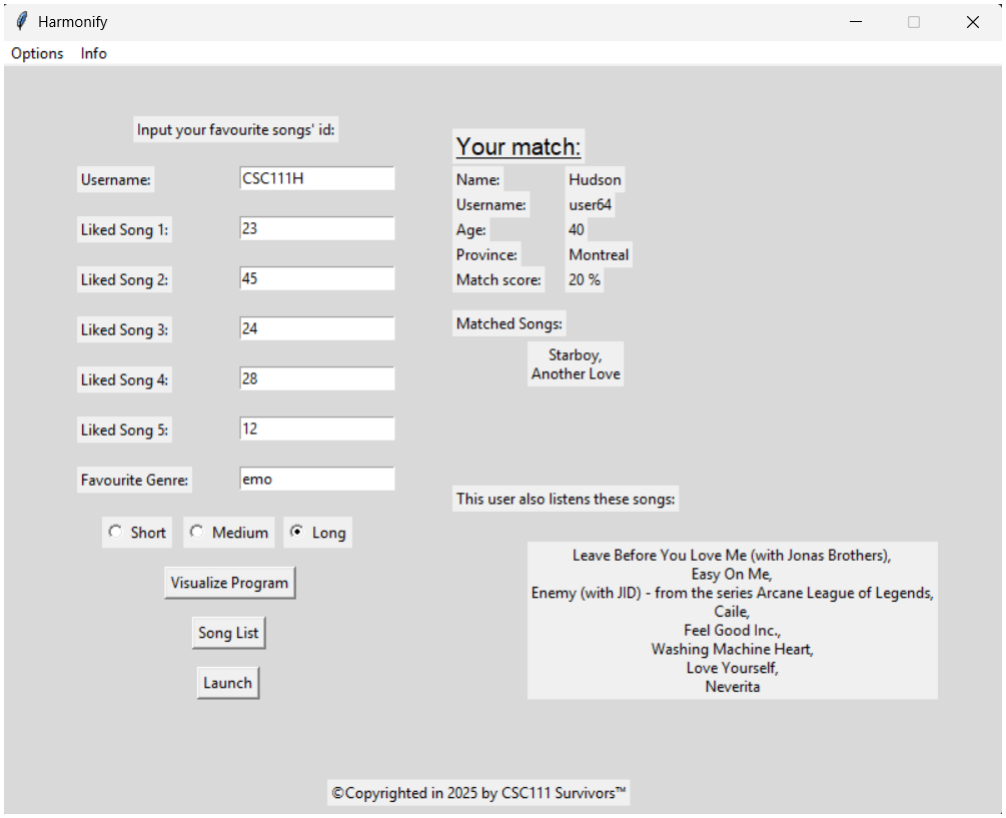

Figure 4: A sample Program interface when the program is launched after the required parameters are entered

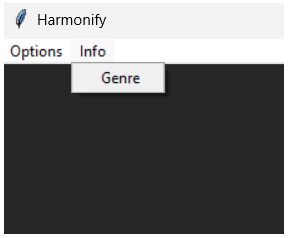

Figure 5: Information section of the program

Here, you can access the available genres to indicate your favorite genre.

Figure 6: Information message where it displays all of the available genres when you select "Genre" on the Info section

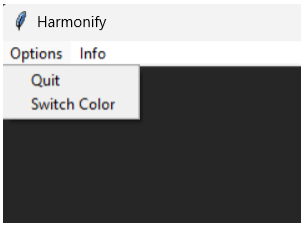

Figure 7: Options menu of our program

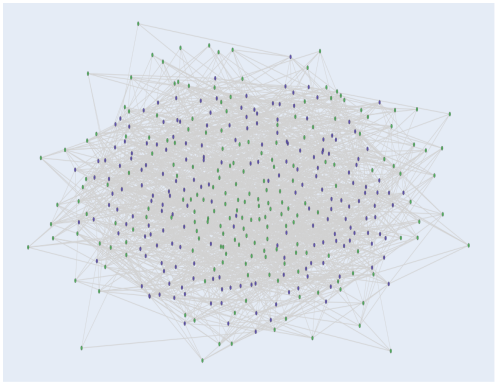

Figure 8: Visualization of our data with graph

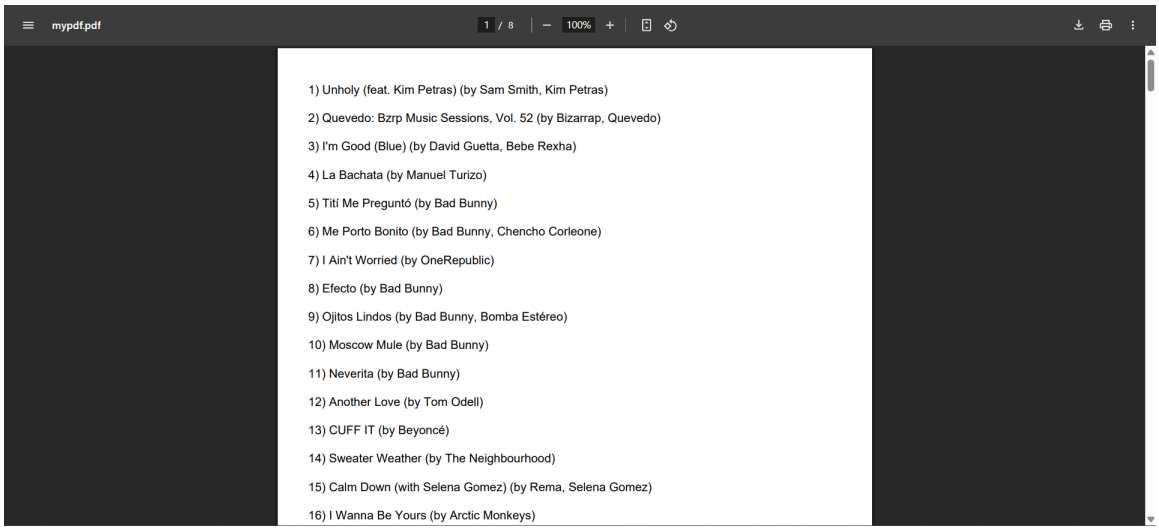

Figure 9: The PDF file that opens when the user selects "Song List"

5 Changes to Original Proposal:

From the original aim proposed on the proposal, the project maintained its goal of recommending songs and users to the input user and it has remained loyal to this goal. However, there were some changes on how Harmonify accomplishes this goal.

1. **Subclass:** The idea of users and songs being a subclass of _Vertex was proposed in the proposal. However, after brief discussion, we have come to conclusion that using "kind" to differentiate between user and song would not only make the code more concise, but also make our computations easier in terms of finding the similarity score and compatible user.

2. **Log-in section:** In the proposal, it was aimed to create a log-in section for the users, and then each user input was going to be added to our graph, and therefore to our csv file. However, due to our limited knowledge on working with tkinter and limited period of conducting this project, we unfortunately were not able to create a log-in section for the users.

3. **Number of songs listed by the user:** The number of songs the user was required to put was reduced from 10 to 5, simply due to the interface adjustments. Creating 10 rows, including a box for favorite genre and adding buttons for the song duration caused our inputs to hold a great amount of area within our user interface, so we have decided to reduce the number of songs listed to 5 due for better user experience.

4. **Song inputs by the user:** Originally on the proposal, we approved that users were going to write down their favourite songs' names but after further discussion we have came to conclusion that writing songs' full names might cause our program to give wrong results since writing down the full song names could result in typos, and thus result in the program not working properly. Therefore, instead of asking users to write down the songs' names they simply just need to write down song's ID which are just integer numbers between 1 and 200. This was provided to the users through the interface. Hence, we have prevented possible wrong entries and made our interface more user-friendly.

5. **Change of computation for compatible user:** In proposal we indicated our compatible user computation as: "We will create a dictionary with every username that maps to a set of songs that they have in common with the user. Using that dictionary, we will create a new dictionary that maps users to the calculated weighted compatibility score. Based on the weighted score dictionary, we will return the (1) **most compatible user that has the highest weighted score** and (2) **the songs that they listen excluding the common songs** to the new user."
However, in our project we decided to use scores as keys that maps to the users instead of using users as key that maps to the scores. This was solely because we can just get the key with the greatest value and get the corresponding user from the dictionary. Furthermore, we didn't create a dictionary that maps every username to the set of songs that have in common with the new user because we found a way that makes this dictionary completely useless.

6 Discussion:

Harmonify aims to connect users across Canada by recommending new songs tailored to their tastes. By bridging distances and enhancing musical discovery, Harmonify offers an up-to-date way for users to engage with music and each other. The process of recommending songs and users was successfully implemented, as stated in our research question.

The service provided by Harmonify removes the barriers created by Spotify and creates a more human-like program, by combining best of both worlds: human interaction and music.

However, there were several drawbacks within our project.

1. First of all, our user dataset was a synthetic data set where the 'Liked Songs' by each synthetic user were randomly generated simply choosing numbers from 1 to 200. This may not entirely reflect the overall sentiment of a user in a real-world context, as although a user can be expected to listen to diverse genres, consistency cannot entirely be seen, or any sort of real-life patterns cannot be observed within our data.

2. Our program did not take the "age" nor the "province" of each user into consideration. A more detailed and complex program would also take into account the preferred age and the nearby provinces, so that these users can meet up in real life.

3. Our limited number of songs also posed a limitation in obtaining more accurate data. If we were to use all the rows in the songs dataset (which contains nearly 90,000 songs), it would not only increase the runtime complexity, but also make it harder to find a match for each user, simply because the probability of finding an exact match would be significantly low.

4. When the program was run, its visual appearance differed between MacOS and Windows. On MacOS, the buttons were shifted and mixed with the textboxes. It is **sincerely** recommended that users run the program on a **Windows** computer to experience an optimized user interface.

5. Our song recommendation system only depends on the 5 song inputs by the user. Therefore, our recommended songs does not entirely mean that these are going to be the songs the user have never listened before. A more complex project would have potentially have access to the user's Spotify account, and take into account all the songs they listen to recommend more unique songs.

Next steps:

- To create a bigger-scale project, our two datasets can be implemented using real-life data instead of synthetic data, with the song data covering all the possible songs, and our user dataset can be obtained through a survey, as mentioned in **2.1**, in User Dataset Part.
- The song dataset can include the other variables of the songs to create a better song recommendation system for the users. This can be done by also taking into account the song's energy, key or loudness, which were included in the original dataset. Including these factors within the preferences could generate more accurate results in terms of finding songs the input user will potentially enjoy listening.
- The user's other incentives such as their age, gender and their current living province can also be taken into account to recommend more compatible users.
- The interface can be developed, with a log-in page showcasing the program logo as well as saving the input user to our graph once they are logged in.

With these next steps, the project can be ground-breaking, visionary, innovative, astonishing, revolutionary, pioneering, game-changing, unprecedented.

7 References

- csv - CSV File Reading and Writing. (n.d.). Python Documentation.
  https://docs.python.org/3/library/csv.html
- pandas documentation — pandas 2.2.3 documentation. (n.d.).
  https://pandas.pydata.org/docs/
- NetworkX 3.4.2 documentation. (n.d.).
  https://networkx.org/documentation/stable/reference/index.html
- tkinter - Python interface to Tcl/Tk. (n.d.). Python Documentation.
  https://docs.python.org/3/library/tkinter.html
- platform - Access to underlying platform's identifying data. (n.d.). Python Documentation.
  https://docs.python.org/3/library/platform.html
- subprocess - Subprocess management. (n.d.). Python Documentation.
  https://docs.python.org/3/library/subprocess.html
- os - Miscellaneous operating system interfaces. (n.d.). Python Documentation.
  https://docs.python.org/3/library/os.html
- Liu, X., Yang, Z., & Cheng, J. (2024, January 24). Music recommendation algorithms based on knowledge graph and multi-task feature learning. Nature.
  https://www.nature.com/articles/s41598-024-52463-z
- Fundamentals, O. (2020, April 6). Fundamentals: What is a Knowledge Graph? Ontotext.
  https://www.ontotext.com/knowledgehub/fundamentals/what-is-a-knowledge-graph/
- Spotify Tracks genre. (2023, November 30). Kaggle.
  https://www.kaggle.com/datasets/thedevastator/spotify-tracks-genre-dataset